

Module 4: Pandas Series

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Introduction to Pandas library
- Understanding Series data structure
- Series creation and manipulation
- Indexing and selection operations
- Basic operations on Series

Topics Covered

1. Introduction to Pandas and its ecosystem
2. Creating Series objects
3. Series attributes and methods
4. Indexing and selection
5. Series operations and arithmetic
6. Handling missing values in Series
7. Series methods for data analysis
8. String operations on Series
9. Date and time operations
10. Series aggregation and statistics
11. Series plotting basics
12. File I/O Operations

Exercises

- Series creation and manipulation exercises
- Data analysis problems using Series

1. Introduction to Pandas and its Ecosystem

In [...

```
1
2     import pandas as pd
3     import numpy as np
4
5     # Check pandas version
6     print(f"Pandas version: {pd.__version__}")
7
```

```

8      # Pandas ecosystem overview
9      print("Pandas is built on top of NumPy and provides:")
10     print("- High-performance data structures")
11     print("- Data analysis tools")
12     print("- Data manipulation capabilities")
13     print("- Time series functionality")
14     print("- Easy data import/export")
15
16     # Key pandas data structures
17     print(f"\nMain pandas data structures:")
18     print(f"1. Series - 1D labeled array")
19     print(f"2. DataFrame - 2D labeled data structure")
20     print(f"3. Index - Immutable array implementing an ordered, sliceable sequence")
21
22     # Import common libraries in pandas ecosystem
23     print(f"\nCommon libraries in pandas ecosystem:")
24     print(f"- NumPy: Numerical computing foundation")
25     print(f"- Matplotlib: Plotting and visualization")
26     print(f"- Seaborn: Statistical data visualization")
27     print(f"- Scikit-learn: Machine learning")
28     print(f"- Jupyter: Interactive computing")

```

2. Creating Series Objects

In [...

```

1      # Creating Series from lists
2      data = [10, 20, 30, 40, 50]
3      series1 = pd.Series(data)
4      print(f"Series from list:\n{series1}")
5
6      # Creating Series with custom index
7      index_labels = ['A', 'B', 'C', 'D', 'E']
8      series2 = pd.Series(data, index=index_labels)
9      print(f"\nSeries with custom index:\n{series2}")
10
11     # Creating Series from dictionary
12     data_dict = {'Apple': 3.5, 'Banana': 2.1, 'Orange': 4.0, 'Grape': 1.8}
13     series3 = pd.Series(data_dict)
14     print(f"\nSeries from dictionary:\n{series3}")
15
16     # Creating Series from NumPy array
17     np_array = np.array([1, 2, 3, 4, 5])
18     series4 = pd.Series(np_array, index=['a', 'b', 'c', 'd', 'e'])
19     print(f"\nSeries from NumPy array:\n{series4}")
20
21     # Creating Series with scalar values
22     series5 = pd.Series(7, index=['x', 'y', 'z'])
23     print(f"\nSeries with scalar value:\n{series5}")
24
25     # Creating Series with different data types
26     mixed_data = [1, 2.5, 'hello', True, None]
27     series6 = pd.Series(mixed_data, index=['int', 'float', 'str', 'bool', 'none'])
28     print(f"\nSeries with mixed data types:\n{series6}")
29     print(f>Data type: {series6.dtype}")
30
31     # Series with date index
32     dates = pd.date_range('2024-01-01', periods=5, freq='D')
33

```

```
34 series7 = pd.Series([100, 110, 105, 120, 115], index=dates)
35 print(f"\nSeries with date index:\n{series7}")
```

3. Series Attributes and Methods

In [...

```
1  # Create a sample Series
2  data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
3  index_labels = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
4  series = pd.Series(data, index=index_labels, name='Sample Data')
5
6  print(f"Series:\n{series}")
7
8  # Basic attributes
9  print(f"\n--- Series Attributes ---")
10 print(f"Name: {series.name}")
11 print(f"Index: {series.index}")
12 print(f"Values: {series.values}")
13 print(f>Data type: {series.dtype}")
14 print(f"Size: {series.size}")
15 print(f"Shape: {series.shape}")
16 print(f"Memory usage: {series.memory_usage()} bytes")
17
18 # Index attributes
19 print(f"\n--- Index Attributes ---")
20 print(f"Index values: {series.index.values}")
21 print(f"Index name: {series.index.name}")
22 print(f"Index is unique: {series.index.is_unique}")
23
24 # Data access methods
25 print(f"\n--- Data Access Methods ---")
26 print(f"First 3 values: {series.head(3)}")
27 print(f>Last 3 values: {series.tail(3)}")
28 print(f"Sample values: {series.sample(3)}")
29
30 # Statistical methods
31 print(f"\n--- Statistical Methods ---")
32 print(f"Count: {series.count()}")
33 print(f"Sum: {series.sum()}")
34 print(f"Mean: {series.mean():.2f}")
35 print(f"Median: {series.median():.2f}")
36 print(f"Mode: {series.mode().values}")
37 print(f"Standard deviation: {series.std():.2f}")
38 print(f"Variance: {series.var():.2f}")
39 print(f"Min: {series.min()}")
40 print(f"Max: {series.max()}")
41 print(f"Range: {series.max() - series.min()}")
42
43 # Data type methods
44 print(f"\n--- Data Type Methods ---")
45 print(f"Is numeric: {series.dtype in ['int64', 'float64']}")
46 print(f"Has null values: {series.isnull().any()}")
47 print(f"Number of null values: {series.isnull().sum()}")
48
49 # Sorting methods
50 print(f"\n--- Sorting Methods ---")
51 print(f"Sorted ascending:\n{series.sort_values()}")
```

```

55 print(f"Sorted descending:\n{series.sort_values(ascending=False)}")
56 print(f"Sorted by index:\n{series.sort_index()}")
57
58 # Unique values
59 print(f"\n--- Unique Values ---")
60 print(f"Unique values: {series.unique()}")
print(f"Number of unique values: {series.nunique()}")
print(f"Value counts:\n{series.value_counts()}")

```

4. Indexing and Selection

In [...

```

1 # Create a comprehensive Series for indexing examples
2 data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
3 index_labels = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
4 series = pd.Series(data, index=index_labels, name='Sample Data')
5
6
7 print(f"Original Series:\n{series}")
8
9 # Position-based indexing (iloc)
10 print(f"\n--- Position-based Indexing (iloc) ---")
11 print(f"First element: {series.iloc[0]}")
12 print(f"Last element: {series.iloc[-1]}")
13 print(f"First 3 elements: {series.iloc[:3]}")
14 print(f"Last 3 elements: {series.iloc[-3:]}")
15 print(f"Elements at positions 1, 3, 5: {series.iloc[[1, 3, 5]]}")
16
17 # Label-based indexing (loc)
18 print(f"\n--- Label-based Indexing (loc) ---")
19 print(f"Element with label 'C': {series.loc['C']}")
20 print(f"Elements with labels 'A', 'C', 'E': {series.loc[['A', 'C', 'E']]}")
21 print(f"Elements from 'B' to 'F': {series.loc['B':'F']}")
22 print(f"Elements from 'D' to end: {series.loc['D':]}")
23
24 # Boolean indexing
25 print(f"\n--- Boolean Indexing ---")
26 print(f"Elements > 50: {series[series > 50]}")
27 print(f"Elements between 30 and 70: {series[(series >= 30) & (series < 70)]}")
28 print(f"Even elements: {series[series % 2 == 0]}")
29
30 # Using .at and .iat for single value access
31 print(f"\n--- Single Value Access ---")
32 print(f"Value at position 2: {series.iat[2]}")
33 print(f"Value at label 'F': {series.at['F']}")
34
35 # Setting values
36 series_copy = series.copy()
37 series_copy.loc['A'] = 15
38 series_copy.iloc[1] = 25
39 print(f"\nAfter modification:\n{series_copy}")

```

5. Series Operations and Arithmetic

```

1  # Create two Series for operations
2
3  s1 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
4  s2 = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
5  s3 = pd.Series([100, 200, 300], index=['a', 'b', 'c']) # Different length
6
7  print(f"Series 1:\n{s1}")
8  print(f"\nSeries 2:\n{s2}")
9  print(f"\nSeries 3 (different length):\n{s3}")
10
11 # Basic arithmetic operations
12 print(f"\n--- Arithmetic Operations ---")
13 print(f"Addition: {s1 + s2}")
14 print(f"Subtraction: {s2 - s1}")
15 print(f"Multiplication: {s1 * s2}")
16 print(f"Division: {s2 / s1}")
17 print(f"Power: {s1 ** 2}")
18
19 # Operations with different indices (alignment)
20 print(f"\n--- Operations with Different Indices ---")
21 print(f"s1 + s3 (aligned):\n{s1 + s3}")
22 print(f"s1 * 10 (scalar):\n{s1 * 10}")
23
24 # Comparison operations
25 print(f"\n--- Comparison Operations ---")
26 print(f"s1 > 2:\n{s1 > 2}")
27 print(f"s1 == s2:\n{s1 == s2}")
28 print(f"s1 >= 3:\n{s1 >= 3}")
29
30 # Logical operations
31 print(f"\n--- Logical Operations ---")
32 mask1 = s1 > 2
33 mask2 = s1 < 5
34 print(f"mask1 (s1 > 2): {mask1}")
35 print(f"mask2 (s1 < 5): {mask2}")
36 print(f"mask1 AND mask2: {mask1 & mask2}")
37 print(f"mask1 OR mask2: {mask1 | mask2}")
38
39 # String operations (if Series contains strings)
40 text_series = pd.Series(['hello', 'world', 'python', 'pandas'], index=[0, 1, 2, 3])
41 print(f"\n--- String Operations ---")
42 print(f"Original: {text_series}")
43 print(f"Uppercase: {text_series.str.upper()}")
44 print(f"Length: {text_series.str.len()}")
45 print(f"Contains 'o': {text_series.str.contains('o')}")
46 print(f"Starts with 'p': {text_series.str.startswith('p')}")
47
48 # Mathematical functions
49 print(f"\n--- Mathematical Functions ---")
50 print(f"Square root: {np.sqrt(s1)}")
51 print(f"Exponential: {np.exp(s1)}")
52 print(f"Logarithm: {np.log(s1)}")
53 print(f"Absolute value: {np.abs(s1 - 3)}")
54
55 # Aggregation functions
56 print(f"\n--- Aggregation Functions ---")
57 print(f"Sum: {s1.sum()}")
58 print(f"Mean: {s1.mean():.2f}")
59 print(f"Standard deviation: {s1.std():.2f}")

```

```
print(f"Min: {s1.min()}")
print(f"Max: {s1.max()}")
print(f"Product: {s1.prod()}")
```

6. Handling Missing Values in Series

In [...

```
1  # Create Series with missing values
2  data_with_nan = [1, 2, np.nan, 4, 5, np.nan, 7, 8, 9, 10]
3  index_with_nan = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
4  series_with_nan = pd.Series(data_with_nan, index=index_with_nan)
5
6  print(f"Series with missing values:\n{series_with_nan}")
7
8  # Check for missing values
9  print(f"\n--- Missing Value Detection ---")
10 print(f"Has any missing values: {series_with_nan.isnull().any()}")
11 print(f"Number of missing values: {series_with_nan.isnull().sum()}")
12 print(f"Missing value positions: {series_with_nan.isnull()}")
13 print(f"Non-missing value positions: {series_with_nan.notnull()}")
14
15 # Drop missing values
16 print(f"\n--- Dropping Missing Values ---")
17 dropped = series_with_nan.dropna()
18 print(f"After dropping NaN values:\n{dropped}")
19
20 # Fill missing values
21 print(f"\n--- Filling Missing Values ---")
22 filled_zero = series_with_nan.fillna(0)
23 print(f"Filled with 0:\n{filled_zero}")
24
25 filled_mean = series_with_nan.fillna(series_with_nan.mean())
26 print(f"Filled with mean ({series_with_nan.mean():.2f}):\n{filled_mean}")
27
28 filled_forward = series_with_nan.fillna(method='ffill')
29 print(f"Forward fill:\n{filled_forward}")
30
31 filled_backward = series_with_nan.fillna(method='bfill')
32 print(f"Backward fill:\n{filled_backward}")
33
34 # Interpolation
35 print(f"\n--- Interpolation ---")
36 interpolated = series_with_nan.interpolate()
37 print(f"Linear interpolation:\n{interpolated}")
38
39 # Operations with missing values
40 print(f"\n--- Operations with Missing Values ---")
41 print(f"Sum (ignores NaN): {series_with_nan.sum()}")
42 print(f"Mean (ignores NaN): {series_with_nan.mean():.2f}")
43 print(f"Count (non-NaN values): {series_with_nan.count()}")
44
45 # Replace specific values
46 print(f"\n--- Replacing Values ---")
47 replaced = series_with_nan.replace([np.nan], 999)
48 print(f"Replaced NaN with 999:\n{replaced}")
49
50 # Working with None values
```

```

53 series_with_none = pd.Series([1, 2, None, 4, 5, None, 7])
54 print(f"\nSeries with None values:\n{series_with_none}")
print(f"None values are treated as NaN: {series_with_none.isnull()}")

```

7. Series Methods for Data Analysis

In [...

```

1  # Create comprehensive Series for data analysis examples
2  np.random.seed(42)
3  data = np.random.normal(100, 15, 100) # Normal distribution with
4  index = pd.date_range('2024-01-01', periods=100, freq='D')
5  sales_data = pd.Series(data, index=index, name='Daily Sales')
6
7
8  # Add some missing values for demonstration
9  sales_data.iloc[10:15] = np.nan
10 sales_data.iloc[50:55] = np.nan
11
12 print(f"Sales Data Series (first 10 values):\n{sales_data.head(10)}")
13 print(f"\nSeries info:")
14 print(f"Length: {len(sales_data)}")
15 print(f>Data type: {sales_data.dtype}")
16 print(f"Memory usage: {sales_data.memory_usage()} bytes")
17
18 # Basic data analysis methods
19 print(f"\n--- Basic Data Analysis Methods ---")
20 print(f"Count (non-null values): {sales_data.count()}")
21 print(f"Sum: {sales_data.sum():.2f}")
22 print(f"Mean: {sales_data.mean():.2f}")
23 print(f"Median: {sales_data.median():.2f}")
24 print(f"Mode: {sales_data.mode().values}")
25 print(f"Standard deviation: {sales_data.std():.2f}")
26 print(f"Variance: {sales_data.var():.2f}")
27 print(f"Min: {sales_data.min():.2f}")
28 print(f"Max: {sales_data.max():.2f}")
29 print(f"Range: {sales_data.max() - sales_data.min():.2f}")
30
31 # Percentile analysis
32 print(f"\n--- Percentile Analysis ---")
33 print(f"25th percentile (Q1): {sales_data.quantile(0.25):.2f}")
34 print(f"50th percentile (Q2/Median): {sales_data.quantile(0.5):.2f}")
35 print(f"75th percentile (Q3): {sales_data.quantile(0.75):.2f}")
36 print(f"90th percentile: {sales_data.quantile(0.9):.2f}")
37 print(f"95th percentile: {sales_data.quantile(0.95):.2f}")
38 print(f"99th percentile: {sales_data.quantile(0.99):.2f}")
39
40 # Skewness and kurtosis
41 print(f"\n--- Distribution Shape ---")
42 print(f"Skewness: {sales_data.skew():.4f}")
43 print(f"Kurtosis: {sales_data.kurtosis():.4f}")
44
45 # Data quality analysis
46 print(f"\n--- Data Quality Analysis ---")
47 print(f"Missing values: {sales_data.isnull().sum()}")
48 print(f"Missing percentage: {(sales_data.isnull().sum() / len(sales_data)):.2f}")
49 print(f"Duplicate values: {sales_data.duplicated().sum()}")
50 print(f"Unique values: {sales_data.nunique()}")
51
52
53

```



```

54 # Rolling statistics
55 print(f"\n--- Rolling Statistics ---")
56 print(f"7-day rolling mean (first 10):\n{sales_data.rolling(window=7).mean().head(10)}")
57 print(f"30-day rolling mean (first 10):\n{sales_data.rolling(window=30).mean().head(10)}")
58 print(f"7-day rolling std (first 10):\n{sales_data.rolling(window=7).std().head(10)}")
59
60 # Expanding statistics
61 print(f"\n--- Expanding Statistics ---")
62 print(f"Expanding mean (first 10):\n{sales_data.expanding().mean().head(10)}")
63 print(f"Expanding sum (first 10):\n{sales_data.expanding().sum().head(10)}")
64
65 # Data transformation methods
66 print(f"\n--- Data Transformation Methods ---")
67 print(f"Log transformation: {np.log(sales_data.dropna()).head(10)}")
68 print(f"Square root transformation: {np.sqrt(sales_data.dropna()).head(10)}")
69 print(f"Standardized values (z-score):\n{(sales_data - sales_data.min()) / (sales_data.max() - sales_data.min()).head(10)}")
70 print(f"Normalized values (0-1):\n{(sales_data - sales_data.min()) / (sales_data.max() - sales_data.min()).head(10)}")
71
72 # Data binning
73 print(f"\n--- Data Binning ---")
74 bins = [0, 80, 100, 120, 200]
75 labels = ['Low', 'Medium', 'High', 'Very High']
76 sales_binned = pd.cut(sales_data, bins=bins, labels=labels, include_lowest=True)
77 print(f"Binned sales data:\n{sales_binned.value_counts()}")
78
79 # Outlier detection
80 print(f"\n--- Outlier Detection ---")
81 Q1 = sales_data.quantile(0.25)
82 Q3 = sales_data.quantile(0.75)
83 IQR = Q3 - Q1
84 lower_bound = Q1 - 1.5 * IQR
85 upper_bound = Q3 + 1.5 * IQR
86 outliers = sales_data[(sales_data < lower_bound) | (sales_data > upper_bound)]
87 print(f"Outliers (IQR method): {len(outliers)} values")
88 print(f"Outlier values: {outliers.values}")
89
90 # Data sampling
91 print(f"\n--- Data Sampling ---")
92 print(f"Random sample (5 values):\n{sales_data.sample(5)}")
93 print(f"Systematic sample (every 10th value):\n{sales_data.iloc[0:100:10]}")
94
95 # Data ranking
96 print(f"\n--- Data Ranking ---")
97 print(f"Ranked values (first 10):\n{sales_data.rank().head(10)}")
98 print(f"Ranked values (descending, first 10):\n{sales_data.rank(ascending=False).head(10)}")

```

8. String Operations on Series

In [...

```

1 # Create Series with string data for string operations
2 names = ['John Smith', 'jane doe', 'BOB WILSON', 'Alice Johnson', 'Mike Brown']
3 ages = [25, 30, 35, 28, 42, 33, 29, 31]
4 cities = ['New York', 'los angeles', 'CHICAGO', 'Houston', 'phoenix', 'San Francisco']
5 emails = ['john@email.com', 'jane.doe@company.org', 'bob.wilson@company.com', 'alice.johnson@company.net', 'mike.brown@company.edu']
6
7 # Create Series objects
8 names_series = pd.Series(names, name='Names')
9 ages_series = pd.Series(ages, name='Ages')
10 cities_series = pd.Series(cities, name='Cities')
11 emails_series = pd.Series(emails, name='Emails')

```



```

10 ages_series = pd.Series(ages, name='Ages')
11 cities_series = pd.Series(cities, name='Cities')
12 emails_series = pd.Series(emails, name='Emails')
13
14 print(f"Names Series:\n{names_series}")
15 print(f"\nCities Series:\n{cities_series}")
16 print(f"\nEmails Series:\n{emails_series}")
17
18 # Basic string operations
19 print(f"\n--- Basic String Operations ---")
20 print(f"Uppercase names:\n{names_series.str.upper()}")
21 print(f"Lowercase names:\n{names_series.str.lower()}")
22 print(f"Title case names:\n{names_series.str.title()}")
23 print(f"Capitalize names:\n{names_series.str.capitalize()}")
24
25 # String length and character operations
26 print(f"\n--- String Length and Character Operations ---")
27 print(f"Length of each name:\n{names_series.str.len()}")
28 print(f"First character of each name:\n{names_series.str[0]}")
29 print(f>Last character of each name:\n{names_series.str[-1]}")
30 print(f"Characters 2-5 of each name:\n{names_series.str[1:5]}")
31
32 # String splitting and joining
33 print(f"\n--- String Splitting and Joining ---")
34 print(f"Split names by space:\n{names_series.str.split(' ')}")
35 print(f"First name only:\n{names_series.str.split(' ').str[0]}")
36 print(f>Last name only:\n{names_series.str.split(' ').str[-1]}")
37 print(f"Join with underscore:\n{names_series.str.replace(' ', '_')}")
38
39 # String searching and pattern matching
40 print(f"\n--- String Searching and Pattern Matching ---")
41 print(f"Names containing 'John':\n{names_series.str.contains('John')}")
42 print(f"Names starting with 'J':\n{names_series.str.startswith('J')}")
43 print(f"Names ending with 'son':\n{names_series.str.endswith('son')}")
44 print(f"Names matching pattern 'J.*':\n{names_series.str.match('J.*')}")
45
46 # String replacement
47 print(f"\n--- String Replacement ---")
48 print(f"Replace 'John' with 'Jonathan':\n{names_series.str.replace('John', 'Jonathan')}")
49 print(f"Replace multiple patterns:\n{names_series.str.replace(['John', 'son'], 'Jonathan', 'son')}")
50 print(f"Remove spaces:\n{names_series.str.replace(' ', '')}")
51
52 # Regular expressions
53 print(f"\n--- Regular Expressions ---")
54 print(f"Extract first word (first name):\n{names_series.str.extract('^(.+)$')}")
55 print(f"Extract last word (last name):\n{names_series.str.extract('([a-zA-Z]*)$')}")
56 print(f"Find all words:\n{names_series.str.findall(r'\w+')}")
57 print(f"Count vowels in each name:\n{names_series.str.count(r'[aeiou]}')}")
58
59 # Email operations
60 print(f"\n--- Email Operations ---")
61 print(f>Email domains:\n{emails_series.str.extract(r'@(.+)$')}")
62 print(f>Email usernames:\n{emails_series.str.extract(r'^(.+)$')}")
63 print(f>Emails from 'email.com':\n{emails_series.str.contains('@email.com')}")
64 print(f"Replace domain:\n{emails_series.str.replace(r'@.*', '@newdomain.com')}")
65
66 # String padding and alignment
67 print(f"\n--- String Padding and Alignment ---")
68 print(f"Pad names to 15 characters (right):\n{names_series.str.pad(15, 'right')}")
69 print(f"Pad names to 15 characters (left):\n{names_series.str.pad(15, 'left')}")

```

```

74 print(f"Center names in 15 characters:\n{names_series.str.center(
75 print(f"Pad with zeros to 10 characters:\n{names_series.str.zfill(
76
77 # String case operations
78 print(f"\n--- String Case Operations ---")
79 print(f"Swap case:\n{names_series.str.swapcase()}")
80 print(f"Is lowercase:\n{names_series.str.islower()}")
81 print(f"Is uppercase:\n{names_series.str.isupper()}")
82 print(f"Is title case:\n{names_series.str.istitle()}")
83
84 # String validation
85 print(f"\n--- String Validation ---")
86 print(f"Is alphabetic:\n{names_series.str.isalpha()}")
87 print(f"Is alphanumeric:\n{names_series.str.isalnum()}")
88 print(f"Is numeric:\n{names_series.str.isnumeric()}")
89 print(f"Is digit:\n{names_series.str.isdigit()}")
90
91 # String slicing with conditions
92 print(f"\n--- String Slicing with Conditions ---")
93 # Create a Series with mixed data types
94 mixed_series = pd.Series(['123abc', '456def', '789ghi', 'abc123',
95 print(f"Mixed series:\n{mixed_series}")
96 print(f"Extract first 3 characters:\n{mixed_series.str[:3]}")
97 print(f"Extract last 3 characters:\n{mixed_series.str[-3:]}")
98 print(f"Extract middle 3 characters:\n{mixed_series.str[1:4]}")
99
100 # String operations with missing values
101 print(f"\n--- String Operations with Missing Values ---")
102 names_with_nan = pd.Series(['John', 'Jane', None, 'Bob', 'Alice'])
103 print(f"Names with NaN:\n{names_with_nan}")
104 print(f"Uppercase (handles NaN):\n{names_with_nan.str.upper()}")
105 print(f"Length (handles NaN):\n{names_with_nan.str.len()}")
106 print(f"Contains 'a' (handles NaN):\n{names_with_nan.str.contains(
107
108 # Advanced string operations
109 print(f"\n--- Advanced String Operations ---")
110 # Create a Series with addresses
111 addresses = pd.Series(['123 Main St, New York, NY 10001', '456 Oa
112 print(f"Addresses:\n{addresses}")
113 print(f"Extract street numbers:\n{addresses.str.extract(r'^(\d+)'
114 print(f"Extract states:\n{addresses.str.extract(r', ([A-Z]{2}) ')}
115 print(f"Extract zip codes:\n{addresses.str.extract(r'(\d{5})$')}")
116 print(f"Split by comma and get first part:\n{addresses.str.split(
117
118 # String operations for data cleaning
119 print(f"\n--- String Operations for Data Cleaning ---")
120 dirty_data = pd.Series([' John Smith ', 'jane.doe', 'BOB-WILSON
121 print(f"Dirty data:\n{dirty_data}")
122 print(f"Strip whitespace:\n{dirty_data.str.strip()}")
123 print(f"Replace dots with spaces:\n{dirty_data.str.replace('.', '
124 print(f"Replace hyphens with spaces:\n{dirty_data.str.replace('-',
125 print(f"Replace @ with space:\n{dirty_data.str.replace('@', ' ')}
126 print(f"Replace underscores with spaces:\n{dirty_data.str.replace(
127 print(f"Clean and title case:\n{dirty_data.str.strip().str.replac

```

9. Date and Time Operations

In [...

```
1  # Create Series with datetime data
2
3  import pandas as pd
4  import numpy as np
5  from datetime import datetime, timedelta
6
7  # Create date range
8  dates = pd.date_range('2024-01-01', periods=30, freq='D')
9  values = np.random.randint(50, 150, 30)
10 date_series = pd.Series(values, index=dates, name='Daily Values')
11
12 print(f"Date Series:\n{date_series.head(10)}")
13 print(f"\nIndex type: {type(date_series.index)}")
14 print(f"Index data type: {date_series.index.dtype}")
15
16 # Basic datetime operations
17 print(f"\n--- Basic Datetime Operations ---")
18 print(f"Year: {date_series.index.year}")
19 print(f"Month: {date_series.index.month}")
20 print(f"Day: {date_series.index.day}")
21 print(f"Day of week: {date_series.index.dayofweek}")
22 print(f"Day name: {date_series.index.day_name()}")
23 print(f"Month name: {date_series.index.month_name()}")
24
25 # Date filtering
26 print(f"\n--- Date Filtering ---")
27 print(f"January 2024 data:\n{date_series[date_series.index.month == 1]}")
28 print(f"First week data:\n{date_series[date_series.index.day <= 7]}")
29 print(f"Weekend data:\n{date_series[date_series.index.dayofweek > 4]}")
30 print(f"Specific date range:\n{date_series['2024-01-10':'2024-01-20']}")
31
32 # Date arithmetic
33 print(f"\n--- Date Arithmetic ---")
34 print(f"Add 5 days to index:\n{date_series.index + timedelta(days=5)}")
35 print(f"Subtract 1 week from index:\n{date_series.index - timedelta(days=7)}")
36 print(f"Add 1 month to index:\n{date_series.index + pd.DateOffset(months=1)}")
37
38 # Time-based resampling
39 print(f"\n--- Time-based Resampling ---")
40 # Create hourly data for resampling example
41 hourly_dates = pd.date_range('2024-01-01', periods=24, freq='H')
42 hourly_values = np.random.randint(10, 100, 24)
43 hourly_series = pd.Series(hourly_values, index=hourly_dates, name='Hourly Values')
44
45 print(f"Hourly data:\n{hourly_series.head()}")
46 print(f"Daily resample (mean):\n{hourly_series.resample('D').mean()}")
47 print(f"Daily resample (sum):\n{hourly_series.resample('D').sum()}")
48 print(f"Daily resample (max):\n{hourly_series.resample('D').max()}")
49
50 # Rolling operations on datetime data
51 print(f"\n--- Rolling Operations on Datetime Data ---")
52 print(f"3-day rolling mean:\n{date_series.rolling('3D').mean().head(3)}")
53 print(f"7-day rolling sum:\n{date_series.rolling('7D').sum().head(7)}")
54 print(f"30-day rolling std:\n{date_series.rolling('30D').std().head(30)}")
55
56 # Expanding operations
57 print(f"\n--- Expanding Operations ---")
58 print(f"Expanding mean:\n{date_series.expanding().mean().head(10)}")
59 print(f"Expanding sum:\n{date_series.expanding().sum().head(10)}")
```

```

63
64 # Date shifting
65 print(f"\n--- Date Shifting ---")
66 print(f"Shift forward by 1 day:\n{date_series.shift(1).head(10)}")
67 print(f"Shift backward by 1 day:\n{date_series.shift(-1).head(10)}")
68 print(f"Shift by 1 week:\n{date_series.shift(7).head(10)}")
69
70 # Period operations
71 print(f"\n--- Period Operations ---")
72 # Convert to period
73 period_series = date_series.to_period('D')
74 print(f"Converted to period:\n{period_series.head()}")
75 print(f"Period frequency: {period_series.index.freq}")
76
77 # Convert to different frequencies
78 monthly_period = date_series.to_period('M')
79 print(f"Monthly period:\n{monthly_period}")
80
81 # Time zone operations
82 print(f"\n--- Time Zone Operations ---")
83 # Create timezone-aware series
84 tz_series = date_series.copy()
85 tz_series.index = tz_series.index.tz_localize('UTC')
86 print(f"UTC timezone:\n{tz_series.head()}")
87
88 # Convert to different timezone
89 est_series = tz_series.tz_convert('US/Eastern')
90 print(f"EST timezone:\n{est_series.head()}")
91
92 # Date formatting
93 print(f"\n--- Date Formatting ---")
94 print(f"Formatted dates (YYYY-MM-DD):\n{date_series.index.strftime('%Y-%m-%d')}")
95 print(f"Formatted dates (Month Day, Year):\n{date_series.index.strftime('%b %d, %Y')}")
96 print(f"Formatted dates (Day/Month/Year):\n{date_series.index.strftime('%d/%m/%Y')}")
97
98 # Business day operations
99 print(f"\n--- Business Day Operations ---")
100 # Create business day series
101 bday_series = pd.Series(np.random.randint(100, 200, 20),
102                          index=pd.bdate_range('2024-01-01', periods=20,
103                                                  name='Business Values'))
104 print(f"Business day series:\n{bday_series.head(10)}")
105
106 # Check if dates are business days
107 print(f"Is business day:\n{date_series.index.isin(pd.bdate_range('2024-01-01', periods=20, name='Business Values'))}")
108
109 # Business day arithmetic
110 print(f"Add 5 business days:\n{date_series.index + pd.offsets.BDay(5)}")
111
112 # Time delta operations
113 print(f"\n--- Time Delta Operations ---")
114 # Create time delta series
115 time_deltas = pd.Series([timedelta(days=1), timedelta(hours=12),
116                          timedelta(seconds=45), timedelta(microseconds=1000000)])
117 print(f"Time deltas:\n{time_deltas}")
118
119 # Operations on time deltas
120 print(f"Total time:\n{time_deltas.sum()}")
121 print(f"Mean time:\n{time_deltas.mean()}")
122 print(f"Convert to seconds:\n{time_deltas.dt.total_seconds()}")

```

```

127
128     # Date comparison
129     print(f"\n--- Date Comparison ---")
130     reference_date = pd.Timestamp('2024-01-15')
131     print(f"Dates before reference:\n{date_series.index < reference_d")
132     print(f"Dates after reference:\n{date_series.index > reference_da")
133     print(f"Dates equal to reference:\n{date_series.index == referenc")
134
135     # Date difference
136     print(f"Days from reference:\n{(date_series.index - reference_dat")
137
138
139     # Seasonal analysis
140     print(f"\n--- Seasonal Analysis ---")
141     # Add seasonal data
142     seasonal_data = np.sin(np.arange(30) * 2 * np.pi / 365) * 10 + 10
143     seasonal_series = pd.Series(seasonal_data, index=dates, name='Sea')
144     print(f"Seasonal data:\n{seasonal_series.head(10)}")
145
146     # Group by month
147     monthly_groups = seasonal_series.groupby(seasonal_series.index.mo)
148     print(f"Monthly means:\n{monthly_groups.mean()}")
149
150     # Group by day of week
151     dow_groups = seasonal_series.groupby(seasonal_series.index.dayofw)
152     print(f"Day of week means:\n{dow_groups.mean()}")
153
154     # Date range operations
155     print(f"\n--- Date Range Operations ---")
156     # Create date range
157     date_range = pd.date_range('2024-01-01', '2024-12-31', freq='M')
158     print(f"Monthly date range:\n{date_range}")
159
160     # Check if dates are in range
161     in_range = date_series.index.isin(date_range)
162     print(f"Dates in monthly range:\n{in_range}")
163
164     # Create custom date range
165     custom_range = pd.date_range('2024-01-01', periods=10, freq='2W')
166     print(f"Bi-weekly custom range:\n{custom_range}")
167
168     # Working with different date formats
169     print(f"\n--- Working with Different Date Formats ---")
170     # String dates
171     string_dates = ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05']
172     string_series = pd.Series([10, 20, 30, 40, 50], index=string_dates)
173     print(f"String dates series:\n{string_series}")
174
175     # Convert string to datetime
176     datetime_index = pd.to_datetime(string_series.index)
177     print(f"Converted to datetime:\n{datetime_index}")
178
179     # Parse different date formats
180     different_formats = ['01/01/2024', '02-Jan-2024', '2024-01-03', '2024-01-04']
181     parsed_dates = pd.to_datetime(different_formats)
182     print(f"Parsed different formats:\n{parsed_dates}")

```

10. Series Aggregation and Statistics

```

1  # Create comprehensive Series for aggregation and statistics exam
2  np.random.seed(42)
3  data = np.random.normal(100, 20, 1000) # Normal distribution
4  categories = np.random.choice(['A', 'B', 'C', 'D'], 1000)
5  dates = pd.date_range('2024-01-01', periods=1000, freq='D')
6
7
8  # Create multiple Series for different examples
9  main_series = pd.Series(data, name='Values')
10 category_series = pd.Series(categories, name='Category')
11 date_series = pd.Series(data, index=dates, name='TimeSeries')
12
13 # Add some missing values for demonstration
14 main_series.iloc[100:110] = np.nan
15 main_series.iloc[500:505] = np.nan
16
17 print(f"Main Series (first 10 values):\n{main_series.head(10)}")
18 print(f"Series length: {len(main_series)}")
19 print(f"Missing values: {main_series.isnull().sum()}")
20
21 # Basic aggregation functions
22 print(f"\n--- Basic Aggregation Functions ---")
23 print(f"Count (non-null): {main_series.count()}")
24 print(f"Sum: {main_series.sum():.2f}")
25 print(f"Mean: {main_series.mean():.2f}")
26 print(f"Median: {main_series.median():.2f}")
27 print(f"Mode: {main_series.mode().values}")
28 print(f"Standard deviation: {main_series.std():.2f}")
29 print(f"Variance: {main_series.var():.2f}")
30 print(f"Min: {main_series.min():.2f}")
31 print(f"Max: {main_series.max():.2f}")
32 print(f"Range: {main_series.max() - main_series.min():.2f}")
33
34 # Percentile statistics
35 print(f"\n--- Percentile Statistics ---")
36 percentiles = [0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99]
37 for p in percentiles:
38     print(f"{p*100:4.0f}th percentile: {main_series.quantile(p):.2f}")
39
40 # Using describe() for comprehensive statistics
41 print(f"\n--- Comprehensive Statistics (describe) ---")
42 print(f"Description:\n{main_series.describe()}")
43
44 # Custom percentiles
45 print(f"\n--- Custom Percentiles ---")
46 custom_percentiles = [0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95]
47 print(f"Custom percentiles:\n{main_series.quantile(custom_percentiles)}")
48
49 # Distribution statistics
50 print(f"\n--- Distribution Statistics ---")
51 print(f"Skewness: {main_series.skew():.4f}")
52 print(f"Kurtosis: {main_series.kurtosis():.4f}")
53 print(f"Kurtosis excess: {main_series.kurtosis() - 3:.4f}")
54
55 # Robust statistics (less sensitive to outliers)
56 print(f"\n--- Robust Statistics ---")
57 print(f"Median absolute deviation: {np.median(np.abs(main_series - main_series.median())):.2f}")
58 print(f"Interquartile range (IQR): {main_series.quantile(0.75) - main_series.quantile(0.25):.2f}")
59

```



```

63 # Grouped aggregation
64 print(f"\n--- Grouped Aggregation ---")
65 # Create a DataFrame for grouped operations
66 df = pd.DataFrame({
67     'values': main_series,
68     'category': category_series
69 })
70
71 print(f"Grouped by category - mean:\n{df.groupby('category')['values'].mean()}")
72 print(f"Grouped by category - std:\n{df.groupby('category')['values'].std()}")
73 print(f"Grouped by category - count:\n{df.groupby('category')['values'].count()}")
74
75 # Multiple aggregations at once
76 print(f"\n--- Multiple Aggregations ---")
77 agg_results = df.groupby('category')['values'].agg(['count', 'mean', 'std'])
78 print(f"Multiple aggregations:\n{agg_results}")
79
80 # Custom aggregation functions
81 print(f"\n--- Custom Aggregation Functions ---")
82 def range_func(x):
83     return x.max() - x.min()
84
85 def coefficient_of_variation(x):
86     return x.std() / x.mean() if x.mean() != 0 else np.nan
87
88 custom_agg = df.groupby('category')['values'].agg([
89     'count', 'mean', 'std', range_func, coefficient_of_variation
90 ])
91 custom_agg.columns = ['Count', 'Mean', 'Std', 'Range', 'CV']
92 print(f"Custom aggregations:\n{custom_agg}")
93
94 # Time series aggregation
95 print(f"\n--- Time Series Aggregation ---")
96 print(f"Daily resampling (mean):\n{date_series.resample('D').mean()}")
97 print(f"Weekly resampling (sum):\n{date_series.resample('W').sum()}")
98 print(f"Monthly resampling (mean):\n{date_series.resample('M').mean()}")
99
100 # Rolling aggregations
101 print(f"\n--- Rolling Aggregations ---")
102 print(f"7-day rolling mean:\n{date_series.rolling(7).mean().head(10)}")
103 print(f"30-day rolling std:\n{date_series.rolling(30).std().head(10)}")
104 print(f"7-day rolling min:\n{date_series.rolling(7).min().head(10)}")
105 print(f"7-day rolling max:\n{date_series.rolling(7).max().head(10)}")
106
107 # Expanding aggregations
108 print(f"\n--- Expanding Aggregations ---")
109 print(f"Expanding mean:\n{date_series.expanding().mean().head(10)}")
110 print(f"Expanding sum:\n{date_series.expanding().sum().head(10)}")
111 print(f"Expanding std:\n{date_series.expanding().std().head(10)}")
112
113 # Weighted aggregations
114 print(f"\n--- Weighted Aggregations ---")
115 # Create weights
116 weights = np.random.uniform(0.5, 1.5, len(main_series))
117 print(f"Weighted mean: {np.average(main_series.dropna(), weights=weights)}")
118
119 # Using pandas weighted mean
120 weighted_series = pd.Series(main_series.values, weights=weights)
121 print(f"Pandas weighted mean: {weighted_series.mean():.2f}")
122

```



```

127 # Conditional aggregations
128 print(f"\n--- Conditional Aggregations ---")
129 print(f"Mean of values > 100: {main_series[main_series > 100].mean()}")
130 print(f"Mean of values < 80: {main_series[main_series < 80].mean()}")
131 print(f"Count of values between 90 and 110: {(main_series >= 90) & (main_series < 110).count()}")
132
133 # Cumulative aggregations
134 print(f"\n--- Cumulative Aggregations ---")
135 print(f"Cumulative sum (first 10):\n{main_series.cumsum().head(10)}")
136 print(f"Cumulative mean (first 10):\n{main_series.expanding().mean().head(10)}")
137 print(f"Cumulative max (first 10):\n{main_series.cummax().head(10)}")
138 print(f"Cumulative min (first 10):\n{main_series.cummin().head(10)}")
139
140
141 # Statistical tests
142 print(f"\n--- Statistical Tests ---")
143 from scipy import stats
144
145 # Normality test
146 shapiro_stat, shapiro_p = stats.shapiro(main_series.dropna()[:100])
147 print(f"Shapiro-Wilk normality test: statistic={shapiro_stat:.4f}, p={shapiro_p:.4f}")
148
149 # Correlation with time
150 time_correlation = main_series.corr(pd.Series(range(len(main_series)), index=main_series.index))
151 print(f"Correlation with time: {time_correlation:.4f}")
152
153 # Outlier detection using IQR
154 Q1 = main_series.quantile(0.25)
155 Q3 = main_series.quantile(0.75)
156 IQR = Q3 - Q1
157 lower_bound = Q1 - 1.5 * IQR
158 upper_bound = Q3 + 1.5 * IQR
159 outliers = main_series[(main_series < lower_bound) | (main_series > upper_bound)]
160 print(f"Outliers (IQR method): {len(outliers)} values")
161 print(f"Outlier percentage: {len(outliers) / len(main_series) * 100:.2f}%")
162
163 # Z-score outlier detection
164 z_scores = np.abs((main_series - main_series.mean()) / main_series.std())
165 z_outliers = main_series[z_scores > 3]
166 print(f"Z-score outliers (>3 std): {len(z_outliers)} values")
167
168 # Summary statistics by groups
169 print(f"\n--- Summary Statistics by Groups ---")
170 # Create more complex grouping
171 df['value_range'] = pd.cut(df['values'], bins=[0, 80, 100, 120, 200],
172                             labels=['Low', 'Medium', 'High', 'Very High'])
173 print(f"Summary by value range:\n{df.groupby('value_range')['values'].agg(['min', 'max', 'mean', 'count'])}")
174
175 # Cross-tabulation
176 print(f"\n--- Cross-tabulation ---")
177 crosstab = pd.crosstab(df['category'], df['value_range'])
178 print(f"Category vs Value Range:\n{crosstab}")
179
180 # Pivot table style aggregation
181 print(f"\n--- Pivot Table Style Aggregation ---")
182 pivot_agg = df.groupby(['category', 'value_range'])['values'].agg(['min', 'max', 'mean', 'count'])
183 print(f"Pivot aggregation:\n{pivot_agg}")
184
185 # Advanced aggregation with custom functions
186 print(f"\n--- Advanced Aggregation with Custom Functions ---")
187 def custom_stats(x):
188     return {'mean': x.mean(), 'std': x.std(), 'count': x.count()}
189
190 
```

```

191         return pd.Series({
192             'count': x.count(),
193             'mean': x.mean(),
194             'std': x.std(),
195             'skew': x.skew(),
196             'kurt': x.kurtosis(),
197             'q25': x.quantile(0.25),
198             'q75': x.quantile(0.75)
199         })

advanced_stats = df.groupby('category')['values'].apply(custom_stats)
print(f"Advanced statistics by category:\n{advanced_stats}")

# Memory usage and performance
print(f"\n--- Memory Usage and Performance ---")
print(f"Series memory usage: {main_series.memory_usage()} bytes")
print(f"Series memory usage (deep): {main_series.memory_usage(deep=True).sum()} bytes")
print(f>Data type: {main_series.dtype}")
print(f"Size: {main_series.size}")
print(f"Shape: {main_series.shape}")

```

11. Series Plotting Basics

In [...

```

1  # Import plotting libraries
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4  import pandas as pd
5  import numpy as np
6
7
8  # Set up plotting style
9  plt.style.use('default')
10 sns.set_palette("husl")
11
12 # Create various Series for plotting examples
13 np.random.seed(42)
14
15 # Time series data
16 dates = pd.date_range('2024-01-01', periods=100, freq='D')
17 time_series = pd.Series(np.cumsum(np.random.randn(100)) + 100, index=dates)
18
19 # Categorical data
20 categories = ['A', 'B', 'C', 'D', 'E']
21 category_counts = pd.Series([25, 30, 20, 15, 10], index=categories)
22
23 # Numerical data
24 numerical_data = pd.Series(np.random.normal(50, 15, 1000), name='Numerical Data')
25
26 # Sales data with trends
27 sales_dates = pd.date_range('2024-01-01', periods=365, freq='D')
28 sales_data = pd.Series(
29     np.sin(np.arange(365) * 2 * np.pi / 365) * 20 + 100 + np.random.randn(365),
30     index=sales_dates, name='Daily Sales')
31
32
33
34 print("Series plotting examples with pandas and matplotlib")
35 print("=" * 50)

```

```

36
37 # Basic line plot
38 print("\n--- Basic Line Plot ---")
39 fig, axes = plt.subplots(2, 2, figsize=(15, 10))
40
41 # Simple line plot
42 time_series.plot(ax=axes[0, 0], title='Time Series Line Plot', co
43 axes[0, 0].set_ylabel('Value')
44 axes[0, 0].grid(True, alpha=0.3)
45
46 # Line plot with custom styling
47 time_series.plot(ax=axes[0, 1], title='Styled Line Plot',
48                 color='red', linestyle='--', marker='o', markersi
49 axes[0, 1].set_ylabel('Value')
50 axes[0, 1].grid(True, alpha=0.3)
51
52 # Bar plot
53 category_counts.plot(kind='bar', ax=axes[1, 0], title='Category B
54 axes[1, 0].set_ylabel('Count')
55 axes[1, 0].tick_params(axis='x', rotation=45)
56
57 # Histogram
58 numerical_data.plot(kind='hist', ax=axes[1, 1], title='Numerical
59                 bins=30, alpha=0.7, color='orange')
60 axes[1, 1].set_ylabel('Frequency')
61
62 plt.tight_layout()
63 plt.show()
64
65 # Different plot types
66 print("\n--- Different Plot Types ---")
67 fig, axes = plt.subplots(2, 3, figsize=(18, 12))
68
69 # Line plot
70 time_series.plot(ax=axes[0, 0], title='Line Plot', color='blue')
71 axes[0, 0].grid(True, alpha=0.3)
72
73 # Bar plot
74 category_counts.plot(kind='bar', ax=axes[0, 1], title='Bar Plot',
75 axes[0, 1].tick_params(axis='x', rotation=45)
76
77 # Horizontal bar plot
78 category_counts.plot(kind='barh', ax=axes[0, 2], title='Horizontal
79
80 # Histogram
81 numerical_data.plot(kind='hist', ax=axes[1, 0], title='Histogram'
82 axes[1, 0].set_ylabel('Frequency')
83
84 # Box plot
85 numerical_data.plot(kind='box', ax=axes[1, 1], title='Box Plot',
86 axes[1, 1].set_ylabel('Value')
87
88 # Density plot (KDE)
89 numerical_data.plot(kind='density', ax=axes[1, 2], title='Density
90 axes[1, 2].set_ylabel('Density')
91
92 plt.tight_layout()
93 plt.show()
94
95 # Time series specific plots

```

```

100 print("\n--- Time Series Specific Plots ---")
101 fig, axes = plt.subplots(2, 2, figsize=(15, 10))
102
103 # Time series with trend
104 sales_data.plot(ax=axes[0, 0], title='Sales Data with Trend', col
105 axes[0, 0].set_ylabel('Sales')
106 axes[0, 0].grid(True, alpha=0.3)
107
108 # Rolling mean
109 rolling_mean = sales_data.rolling(window=30).mean()
110 sales_data.plot(ax=axes[0, 1], title='Sales with 30-day Rolling M
111 rolling_mean.plot(ax=axes[0, 1], color='red', linewidth=2, label=
112 axes[0, 1].legend()
113 axes[0, 1].grid(True, alpha=0.3)
114
115 # Seasonal decomposition (simplified)
116 monthly_avg = sales_data.groupby(sales_data.index.month).mean()
117 monthly_avg.plot(kind='bar', ax=axes[1, 0], title='Monthly Averag
118 axes[1, 0].set_xlabel('Month')
119 axes[1, 0].set_ylabel('Average Sales')
120 axes[1, 0].tick_params(axis='x', rotation=0)
121
122 # Weekly pattern
123 weekly_avg = sales_data.groupby(sales_data.index.dayofweek).mean(
124 weekdays = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
125 weekly_avg.index = weekdays
126 weekly_avg.plot(kind='bar', ax=axes[1, 1], title='Weekly Sales Pa
127 axes[1, 1].set_xlabel('Day of Week')
128 axes[1, 1].set_ylabel('Average Sales')
129 axes[1, 1].tick_params(axis='x', rotation=45)
130
131 plt.tight_layout()
132 plt.show()
133
134 # Statistical plots
135 print("\n--- Statistical Plots ---")
136 fig, axes = plt.subplots(2, 2, figsize=(15, 10))
137
138 # Histogram with KDE
139 numerical_data.plot(kind='hist', ax=axes[0, 0], title='Histogram
140 bins=30, alpha=0.7, density=True, color='skybl
141 numerical_data.plot(kind='density', ax=axes[0, 0], color='red', l
142 axes[0, 0].set_ylabel('Density')
143
144 # Box plot
145 numerical_data.plot(kind='box', ax=axes[0, 1], title='Box Plot',
146 axes[0, 1].set_ylabel('Value')
147
148 # Q-Q plot (quantile-quantile plot)
149 from scipy import stats
150 stats.probplot(numerical_data.dropna(), dist="norm", plot=axes[1,
151 axes[1, 0].set_title('Q-Q Plot (Normal Distribution)')
152 axes[1, 0].grid(True, alpha=0.3)
153
154 # Cumulative distribution
155 cumulative = numerical_data.sort_values().cumsum() / numerical_da
156 cumulative.plot(ax=axes[1, 1], title='Cumulative Distribution', c
157 axes[1, 1].set_ylabel('Cumulative Probability')
158 axes[1, 1].grid(True, alpha=0.3)
159
160
161
162
163

```

```

164 plt.tight_layout()
165 plt.show()
166
167 # Custom styling and annotations
168 print("\n--- Custom Styling and Annotations ---")
169 fig, ax = plt.subplots(1, 1, figsize=(12, 6))
170
171 # Plot with custom styling
172 time_series.plot(ax=ax, title='Custom Styled Time Series Plot',
173                  color='darkblue', linewidth=2, marker='o', marker
174
175 # Add annotations
176 max_val = time_series.max()
177 max_date = time_series.idxmax()
178 ax.annotate(f'Peak: {max_val:.2f}',
179             xy=(max_date, max_val),
180             xytext=(10, 10),
181             textcoords='offset points',
182             bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow'
183             arrowprops=dict(arrowstyle='->', connectionstyle='arc3
184
185 # Add trend line
186 z = np.polyfit(range(len(time_series)), time_series.values, 1)
187 p = np.poly1d(z)
188 ax.plot(time_series.index, p(range(len(time_series))), "r--", alp
189 ax.legend()
190 ax.grid(True, alpha=0.3)
191 ax.set_ylabel('Value')
192
193 plt.tight_layout()
194 plt.show()
195
196 # Subplot examples
197 print("\n--- Subplot Examples ---")
198 fig, axes = plt.subplots(3, 1, figsize=(12, 15))
199
200 # Original data
201 sales_data.plot(ax=axes[0], title='Original Sales Data', color='b
202 axes[0].set_ylabel('Sales')
203 axes[0].grid(True, alpha=0.3)
204
205 # Moving averages
206 sales_data.plot(ax=axes[1], title='Sales with Moving Averages', c
207 sales_data.rolling(window=7).mean().plot(ax=axes[1], color='red',
208 sales_data.rolling(window=30).mean().plot(ax=axes[1], color='gree
209 axes[1].legend()
210 axes[1].set_ylabel('Sales')
211 axes[1].grid(True, alpha=0.3)
212
213 # Detrended data
214 detrended = sales_data - sales_data.rolling(window=30).mean()
215 detrended.plot(ax=axes[2], title='Detrended Sales Data', color='p
216 axes[2].set_ylabel('Detrended Sales')
217 axes[2].grid(True, alpha=0.3)
218
219 plt.tight_layout()
220 plt.show()
221
222 # Plot customization options
223 print("\n--- Plot Customization Options ---")
224

```

```

228 fig, ax = plt.subplots(1, 1, figsize=(12, 6))
229
230 # Create a more complex plot
231 x = pd.date_range('2024-01-01', periods=50, freq='D')
232 y1 = np.sin(np.arange(50) * 2 * np.pi / 50) * 10 + 50
233 y2 = np.cos(np.arange(50) * 2 * np.pi / 50) * 10 + 50
234
235 series1 = pd.Series(y1, index=x, name='Series 1')
236 series2 = pd.Series(y2, index=x, name='Series 2')
237
238 # Plot multiple series
239 series1.plot(ax=ax, color='blue', linewidth=2, label='Series 1',
240 series2.plot(ax=ax, color='red', linewidth=2, label='Series 2', m
241
242 # Customize the plot
243 ax.set_title('Multiple Series Plot with Custom Styling', fontsize
244 ax.set_xlabel('Date', fontsize=12)
245 ax.set_ylabel('Value', fontsize=12)
246 ax.legend(fontsize=10, loc='upper right')
247 ax.grid(True, alpha=0.3, linestyle='--')
248 ax.set_facecolor('lightgray')
249
250 # Add text annotation
251 ax.text(0.02, 0.98, 'Custom Plot Example', transform=ax.transAxes
252         fontsize=10, verticalalignment='top',
253         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8)
254
255 plt.tight_layout()
256 plt.show()
257
258 # Save plots
259 print("\n--- Saving Plots ---")
260 # Create a simple plot to save
261 fig, ax = plt.subplots(1, 1, figsize=(10, 6))
262 time_series.plot(ax=ax, title='Time Series Plot', color='blue', l
263 ax.grid(True, alpha=0.3)
264 ax.set_ylabel('Value')
265
266 # Save the plot
267 plt.savefig('time_series_plot.png', dpi=300, bbox_inches='tight')
268 print("Plot saved as 'time_series_plot.png'")
269
270 plt.show()
271
272 # Plot parameters and options
273 print("\n--- Plot Parameters and Options ---")
274 print("Common plot parameters:")
275 print("- kind: 'line', 'bar', 'barh', 'hist', 'box', 'density', '
276 print("- color: color name or hex code")
277 print("- title: plot title")
278 print("- xlabel, ylabel: axis labels")
279 print("- figsize: tuple (width, height)")
280 print("- alpha: transparency (0-1)")
281 print("- linewidth: line thickness")
282 print("- marker: marker style ('o', 's', '^', etc.)")
283 print("- markersize: marker size")
284 print("- grid: True/False for grid lines")
285 print("- legend: True/False for legend")
286
287 # Advanced plotting with seaborn

```

```

292 print("\n--- Advanced Plotting with Seaborn ---")
293 fig, axes = plt.subplots(2, 2, figsize=(15, 10))
294
295 # Convert Series to DataFrame for seaborn
296 df_plot = pd.DataFrame({'values': numerical_data, 'category': np.
297
298 # Histogram with seaborn
299 sns.histplot(data=df_plot, x='values', ax=axes[0, 0], kde=True)
300 axes[0, 0].set_title('Seaborn Histogram with KDE')
301
302 # Box plot with seaborn
303 sns.boxplot(data=df_plot, x='category', y='values', ax=axes[0, 1])
304 axes[0, 1].set_title('Seaborn Box Plot by Category')
305
306 # Violin plot
307 sns.violinplot(data=df_plot, x='category', y='values', ax=axes[1,
308 axes[1, 0].set_title('Seaborn Violin Plot by Category')
309
310 # Time series with seaborn
311 ts_df = pd.DataFrame({'date': time_series.index, 'value': time_se
312 sns.lineplot(data=ts_df, x='date', y='value', ax=axes[1, 1])
313 axes[1, 1].set_title('Seaborn Time Series Plot')
314 axes[1, 1].tick_params(axis='x', rotation=45)
315
316 plt.tight_layout()
317 plt.show()
318
319 print("\nSeries plotting examples completed!")
320 print("Key takeaways:")
321 print("1. Use .plot() method for quick plotting")
322 print("2. Customize with matplotlib parameters")
323 print("3. Use seaborn for statistical plots")
324 print("4. Save plots with plt.savefig()")
325 print("5. Combine multiple plots with subplots")

```

Summary

This module covered comprehensive examples for all 11 topics in Pandas Series:

1. **Introduction to Pandas and its ecosystem** - Overview of pandas library and its data structures
2. **Creating Series objects** - Various ways to create Series from different data sources
3. **Series attributes and methods** - Understanding Series properties and basic methods
4. **Indexing and selection** - Accessing data using different indexing methods
5. **Series operations and arithmetic** - Mathematical and logical operations on Series
6. **Handling missing values in Series** - Detection, removal, and filling of missing data
7. **Series methods for data analysis** - Advanced analytical methods and statistical functions
8. **String operations on Series** - Text processing and manipulation using .str accessor

- 9. **Date and time operations** - Working with datetime data and time series
- 10. **Series aggregation and statistics** - Comprehensive statistical analysis and grouping
- 11. **Series plotting basics** - Visualization using matplotlib and seaborn

Key Takeaways:

- Series is a fundamental 1D data structure in pandas
- Powerful indexing and selection capabilities
- Rich set of methods for data manipulation and analysis
- Excellent integration with NumPy and matplotlib
- Essential for data cleaning, analysis, and visualization

Next Steps:

- Practice with real-world datasets
- Explore DataFrame operations (Module 5)
- Learn advanced data manipulation techniques
- Apply Series operations in data science projects